

DATA STRUCTURES AND ALGORITHMS

BSE : 3rd
SEMESTER

23/1/24

- ④ Data Storage, Representation and Retrieve.
- ④ Data Structures in the form of class (organized form).
- ④ Array is a data structure.
(String, date is also data structure).
- ④ Algorithm (Step by step procedure).
(code is algorithm to solve problem)
- ④ Date:

class
{ DD
MM
YYYY
}

3/11/2024
D M Y

- ④ Problems to be solved efficiently. (Timely)

◎ Data Structure:

Data structure is a way of collecting and organizing

data in such a way that we can perform operations on these data in an effective way.

→ Data Structures are the programmatic way of storing data so that data can be used efficiently or effectively.

① Algorithm:

A collection of steps to solve a particular problem. It takes input, process input and gives output.

- (*) ① Take two numbers
- ② Add the number
- ③ Output the final result.

Properties

Any algorithm possess these properties:

- ①(*) Input: Algorithm should have some input

: class structure
variables, template class
(Learn before lab).

②* At least one output should be returned after completion of task.

③* Definiteness: Every statement of the algorithm should be unambiguous.

④* Finiteness: After finite number of steps program should terminate.

⑤* Effectiveness: Every step of the algorithm should be feasible that it is possible to perform each step in a finite time.

⊙ Abstract Data Type (ADT)

Data structure is programmatically version of ADT.

* Complex data type:

$$Z = x + iy$$

$$i = \sqrt{-1}, i^2 = -1$$

{ → Domain → Operations

→ Axioms (Rules) }

(Major three aspects of data structure.

→ User enters imaginary number

and real number. We convert it into complex form.

④ Complex Z

{
 domain { X real
 Y real
 }

operations

0 = ⊕
 ⊖
 ⊗
 conjugate = ∇
 ||
 magnitude

⊙ Axiom 1:

$$z_1 + z_2$$

$$z_1 = x_1 + iy_1$$

$$z_2 = x_2 + iy_2$$

$$z_1 + z_2 = z_3 = (x_1 + x_2) + i(y_1 + y_2)$$

Add-complex (z₁ complex, z₂ complex)

{ Complex z₃;
 z₃.x = z₁.x + z₂.x;

$$z_3.y = z_1.y + z_2.y;$$

return z₃;

}

Display-complex ()

{
 cout << x << "+" << "i"
 << y << endl;

}

Minus-complex (z₁ complex, z₂ complex)

{ complex
z₃;

$$z_3 \cdot x = z_1 \cdot x - z_2 \cdot x;$$

$$z_3 \cdot y = z_1 \cdot y - z_2 \cdot y;$$

} return z₃;

⊙ Axiom 2:

z₁ ⊗ z₂

$$(x_1 x_2 - y_1 y_2) + i(x_1 y_2 + x_2 y_1)$$

Multiply-complex (z₁ complex, z₂ complex)

{ complex z₃,

$$z_3 \cdot x = ((z_1 \cdot x * z_2 \cdot x) - (z_1 \cdot y * z_2 \cdot y))$$

$$z_3 \cdot y = (z_1 \cdot x * z_2 \cdot y) + (z_2 \cdot x * z_1 \cdot y)$$

} return z₃;

* Axiom 3:

z₁ ÷ z₂

$$\left(\frac{x_1 x_2 + y_1 y_2}{x_2^2 + y_2^2} \right) + i \left(\frac{x_2 x_1 - x_1 y_2}{x_2^2 + y_2^2} \right)$$

: sqrt functions
: at least one quiz
every week.

Divide-Complex (z_1 complex, z_2 complex)
Complex z_3 ;

$$z_3 \cdot x = \frac{(z_1 \cdot x * z_2 \cdot y) + (z_1 \cdot y + z_2 \cdot y)}{(z_2 \cdot x)(z_2 \cdot x) + (z_2 \cdot y)(z_2 \cdot y)}$$

$$z_3 \cdot y = (z_2 \cdot x * z_1 \cdot y) -$$

⊛ Axiom 4:

$$\odot \text{Conjugate} = \nabla z_1 = \frac{x_1}{x_1^2 + y_1^2} + i \frac{y_1}{x_1^2 + y_1^2}$$

$$|z| = \sqrt{x_1^2 + y_1^2} \quad : \text{sqrt}(x_1^2 + y_1^2)$$

25/1/24

⊛ Domain (Primitive Data type).

⊛ Algorithm performance analysis (feasible)

Space
Complexity
(memory consumption)

Time
Complexity
(Time consumption)

→ Less space and less time
is an efficient program.

* Space complexity of an algorithm is the total space taken by algorithm with respect to input size.

* Time complexity is the total time taken by algorithm to complete its execution.

- *
① Instruction Space ✓
② Environmental ^{stack} Space x
③ Data space (Input + ✓
Auxillary space) ✓

* Data space depends upon input data type.

- | | |
|-------------|--------------|
| ① char - 1 | ① short - 2 |
| ① int - 4 | ① long -- |
| ① float - 4 | ① Double - 8 |

* Sum (int a, int b, int c)

{
 int z = a+b+c;

 return z;

 a b c z = 16 B
 4 4 4 4

```
int sum(int a[], int N)
```

```
{ int sum, i=0;
```

```
  for (i=0; i<N; i++)
```

```
    { sum = sum + a[i];
```

```
  } return sum;
```

: Complexity depends on N

a = N x 4

N: 4

sum: 4

i: 4

④ Basic Time Complexity is focused. In paper

Complexity mean time complexity. = $O(N)$

(Space Complexity)

④ Time Complexity:

Compile Time
(Donot depend upon instance (input) characters)

Run Time (It is mainly focused)
(Depend on input)

+ , - , *

Time required to perform operations.

④ Method to find Time Complexity

- ① Seconds count of different operations.
- ② Step-count

```

(*) int sum(int *a, const int N)
{
    int s = 0;
    for (int i = 0; i < N; i++)
    {
        s = s + a[i];
    }
    return s;
}

```

$$= 0 + 1 + N + 1 + N + 1$$

$$= 2N + 3 \text{ (Time Complexity).}$$

```

(*) int Abc(int a, int b, int c, int d)
{
    return a * b * (c - d) + 4 + (a * d);
}

```

Big O(1) $\} = 0 + 1 = 0 = 1$ - constant : Constant Complexity (Not depends)

(*) 3 Arrays, 2 Array into : Linear Complexity (Depends on input variable)

```

(*) void add (int **a, int **b,
              int **c, int m, int n)
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    }
}

```

$$= m+1 + m(n+1) + mn$$

$$= m+1 + mn + m + mn$$

Big $O(mn)$

$$= (2m + 2mn + 1)$$

Time Complexity

⊙ If $m=n$:

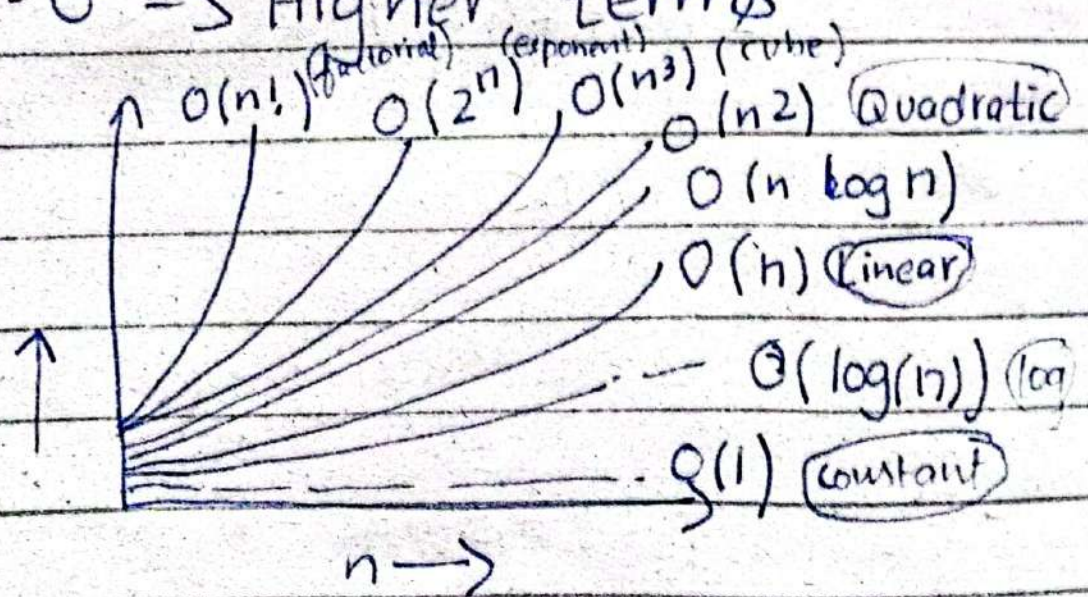
$$= n+1 + n(n+1) + n(n)$$

$$= n+1 + n^2 + n + n^2$$

Big $O(n^2)$

$$= (2n^2 + 2n + 1) \rightarrow \text{Quadratic Complexity}$$

⊙ Big - $O \rightarrow$ Higher Terms



⊙ Multiply two matrix:

```
void multiplyMatrix(int**m, int**n,
                    int**r, int mRows,
                    int mColumns,
                    int nRows, int nColumns)
```

```
{
    for (int i=0; i < mRows; i++) {
        for (int j=0; j < nColumns; j++) {
```

mRows + 1
mRows + 1

30/1/24

⊙ Logarithmic Complexity

⊙ Quadratic Complexity

for (int j=0; j<=n; j*=2) — $\log(n)+1$

{ for (j=1; j<=n; j++) —

④ for (int j, i=1; i<=n; i++) — (n+1)

{ for (j=1; j<=i; j++) — $\frac{n(n+1)}{2}$

{ cout << —

depend on
series

} } : $O(n^2)$

for (cnt=0, i=1; i<=n; i*=2) $\left(\frac{\log(n)}{\log(2)} + 1\right)$

{ for (j=1; j<=i; j++) — $(N-1)/(\log N)$

{ cnt++;

}

$$1 + 2 + 4 + \dots + \log n$$

$$= 2^{\log n}$$

$$\leq 2 \log n = N-1$$

$$n=3$$

$$1, 2, 4$$

$$3-1$$

⑤ Series:

$$(1) \sum_{k=1}^N k = \frac{k(k+1)}{2}$$

$$(2) \sum_{k=1}^N k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$(3) \sum_{n=0}^k x^n = 1 + x + x^2 + x^3 + \dots$$

$$(4) 1 + 2^1 + 2^2 + 2^3 + \dots + 2^n = \sum_{n=0}^k 2^n$$

④ Algorithm to find element (Linear, Algorithm) search

```
int findElement (int * arr, int N,  
                int element)
```

```
{
```

```
    for (int i=0; i<N; i++) — (N+1)
```

```
    { if (arr[i] == element) — (size)
```

```
    {
```

```
        return i;
```

```
    }
```

```
    return -1 — (1)
```

```
}
```

Worst
Case
check for
Big O

⑤ Algorithm Binary search. ^(Sorted Array) (Find from mid)

```
int Binarysearch (int * a, int X)
```

```
{
```

```
    int left=0, right=size, mid;
```

```
    while (left <= right)
```

```
    { mid = (left + right) / 2;
```

```
      if (a[mid] == X)
```

```
      { return mid;
```

```

else if ( a[mid] > x )
{
    right = mid - 1;
}
else
{
    left = mid + 1;
}
}
return n - 1;
}

```

① In-it Print (int n)

```

{
    n = 1000;
    Print n;
}

```

: Constant
Complexity

In-it Print (int n)

```

{
    n = 1000

```

```

    for (int i = 0; i < 4; i++) : Not depend

```

```

    {
        Print n;

```

: Constant
Complexity

① * Next Class Quiz

* Polynomials ADT:

$$P_1 = 4x^3 + 3x^2 + 2$$

:(coefficient,
exponent)

$$P_2 = 3x^2 + 3$$

$$a_0x^{e_0} + a_1x^{e_1} + a_2x^{e_2} \dots a_nx^{e_n}$$

Operations:

① Add ② Subtract

③ Multiply

④ Evaluate (int type)

→ $P_1 + P_2$:

$$P_1 = 4x^3 + 3x^2 + 2$$

$$P_2 = 3x^2 + 3$$

$$= 4x^3 + 6x^2 + 5$$

zero
coefficient
term not
display
($2x^0 = 2$)

$$a(x) = \sum a_i x^{e_i}, b(x) = \sum b_j x^{e_j}$$

$$a+b = \sum (a_i + b_j) x_i$$

$$a-b = \sum (a_i - b_j) x_i$$

$$a \times b = \sum (a_i x^{e_i} \cdot \sum (b_j \cdot x^{e_j}))$$

④ Evaluate:

$$4x^3 + 3x^2 + 2 \quad : x=2$$

$$4(2)^3 + 3(2)^2 + 2$$

$$= 32 + 12 + 2 = 46$$

④ We use array index as exponent.

⑤ Degree of polynomial (maximum power) : degree

: coefficient [Maxdegree + 1]

: Maxdegree = n

⑥ $P_1 = 4x^3 + 3x^2 + 2$

4	3	0	2
---	---	---	---

0 1 2 3 ↘ variable

: nth term =

$(n-i)$

⑦ Display:

coefficient not displays variable not displays.

: degree = n

```
for (int i=0; i<=n; i=i+1)
```

```
{ cout << a[i] << "x" << n-i << "+";
```

```
.. }
```

⑧ Polynomial add (Polynomial p)

```
{ if (this->degree > poly->degree)
```

```
{ max-poly = this;  
  min-poly = poly;
```

```
}
```

else

max.poly = poly;

min.poly = this;

for (int i = 0; i < max.poly.degree; i++)

{
max.poly[i] = max.poly[i] + min.poly[i];
}

$$4x^3 + 6x^2 + 5$$

1 2 3 4
0 6 0 5

④ for (i = 0; i <= maxpoly.degree; i++)

{ res.poly[i] = maxpoly[i] + minpoly[i];
}

⑤ for (int i = 0; i <= max.poly.degree; i++)

{ res.poly[i] = maxpoly[i];
}

for (int i = 0; i <= min.poly.degree; i++)

{ res.poly[i] = res.poly[i] + min.poly[i];
}

(*) $i = \text{max-degree} - \text{min-degree}$

13/2/24

* Polynomial:

P_1 $4x^3 + 2x^2 + 2x + 1$

$d=3$

(max degree)
: Coeff []
Max Degree
degree

P_2 $6x^4 + 5x^2$

$d=4$

	0	1	2	3
P_1	4	2	2	1

exp = 3 2 1 0
(n-i)

	0	1	2	3
P_2	6	0	5	

exp = 4 3 2
(n-i)

: a_{n-i}

Res =

6	0	5	2	1
---	---	---	---	---

 =

6	4	7	2	1
---	---	---	---	---

exp 4 3 2 1 0 exp 4 3 2 1 0

degree = n

(*) Reverse-Order:

	0	1	2	3
P_1	1	2	2	4

Min-degree

	0	1	2	3	4
P_2	0	0	5	0	6

Max-degree

Res = max-degree

⊙ Add-Poly (Polynomial poly)

{ y (degree < poly.degree)

{ Max-degree = poly;
Min-degree = this;

else

{ Max-degree = this;
Min-degree = poly;

```

for (int i=0; i <= Mindegree.degree; i++)
{
    Res.poly[i] = Res.poly[i] + Min.degree[i]
}

```

0	1	2	3	4
0	0	5	0	6

1	2	7	4	6
---	---	---	---	---

→ Resultant

① $P_1 = 4x^3 + 3x^2 + 1$
 $6x^4 + 5x$

$$24x^7 + 12x^6 + 6x^4 + 20x^4 + 10x^3 + 5x$$

$$24x^7 + 12x^6 + 26x^4 + 10x^3 + 5x$$

Multiply - Poly (Polynomial poly)

```

{
    for (int i=0; i < min.degree; i++)

```

```

    {
        Respoly[i] = 0;

```

```

        for (int j=0; j < max.degree; j++)

```

```

        {
            Respoly[i+j] = Respoly[i+j] +

```

(min.degree + max.degree);

② Res-Poly = new Polynomial;

Res-Poly.degree = this.degree + poly.degree;

```

for (i=0; i < this.degree; i=i+1)
{
    for (j=0; j < poly.degree; j=j+1)
    {
        Res.poly[i+j] = Res.poly[i+j] +
            (this.coeff[i] *
             poly.coeff[j]);
    }
}

```

$\therefore O(n^2 + n)$
 $\star \boxed{O(n^2)}$

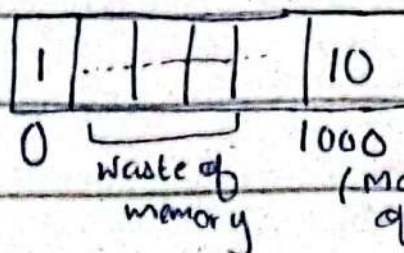
④ Evaluate

$$4x^3 + 2x^2 + 1$$

$$x=2$$

$$4(2)^3 + 2(2)^2 + 1$$

: Drawback
 of Array
 data
 in polynomial



$100x^{1000} + 1$

(make objects of coefficient and exponents).

④ Term

coeff
exp

④ class

```

Polynomial
{
    Term TermArray[];
    int capacity;
    int Term;
}

```

$$4x_0^3 + 2x_1^2 + 1$$

coeff	4	2	1
exp	3	2	0

20/2/24

: Term

{ coefficient;
exp;

: Polynomial

{ Term Term Array[];
capacity;
term;

→ $2x^3 + 6x^2 + 1$

coeff	2	6	1
exp	3	2	0

→ $7x^4 - 2x^3$

coeff	7	-2
exp	4	3

: Linear Big O

: Order

: sparse



→ Polynomial Add (Polynomial b)

{ Polynomial c;

c.term = 0;

c.capacity = a.capacity + b.capacity;
(term) (term)

```

    C.TermArray = new Term[C.capacity];
    for (int i = 0; i < a.Term && i < b.Term; i++)
    {
        if (a.TermArray[i].exp == b.TermArray[i].exp)
        {
            C.TermArray[i].coefficient = a.TermArray[i].coefficient + b.TermArray[i].coefficient;
            C.TermArray[i].exp[i] = a.TermArray.coefficient.exp[i];
            j++;
        }
    }
}

```

```

→ while (apos < Term && bpos < b.Term)
{
    TermArray[apos].cap = b.TermArray[bpos].cap;
    {
        C.TermArray[i] = new Term
        (
            TermArray[apos].coefficient +
            b.TermArray[bpos].coefficient,
            TermArray[apos].exp
        );
        i++, apos++, bpos++
    }
}

```

else if (TermArray[a_pos].exp < b.TermArray[b_pos].exp)

{ C.TermArray[i] = new Term
(b.TermArray[b_pos].coeff,
b.TermArray[b_pos].exp);
}

→ for(i; a_pos < Term; a_pos++)

{ C.TermArray[i] = TermArray[i];
i++;
}

for(j; b_pos < b.Term; b_pos++)

{ C.TermArray[i] = b.TermArray[b_pos];
i++;
} : Arranging.

$$\begin{array}{rcl} \rightarrow & 2x^3 - 10x^2 + 15 & - P_1 \\ & x^2 - 3x + 2 & - P_2 \end{array}$$

2	-10	15
3	2	0

1	-3	2
2	1	0

Dry Run

$$a\ pos = 0$$

$$Term = 3$$

$$b\ pos = 0$$

$$Term = 3$$

Term Array[a pos] . cap =

$i = 0$

C. Term Array[i] = $\downarrow$ a pos

$$P_1 = 2x^3 - 10x^2 + 15$$

2	-10	15
3	2	0

$$P_2 = x^2 - 3x + 2$$

1	-3	2
2	1	0

$\uparrow$ b pos

2	-9	-3	17
3	2	1	0

→ Int Evaluate (int x)

```
{ int res = 0;
```

```
  for ( i = 0 ; i < degree ; i++ )
```

```
  {
```

```
    res = res + this->coeff [i] * pow(x, i);
```

```
  }
```

```
  return res;
```

```
}
```

22/2/24

⚡ (*) Access each object without declaration.

Classical Data Structures

Data Structure

Linear D.S

Non-Linear D.S

Detailed study

→ Array

→ Stack

→ Queue

→ Linked List

→ Trees

→ Graphs

→ Tables

→ Sets

→ **Array:**

Return 4th member → $A[4]$

0	1	2				
6	5	4	3	2	1	0

$A[2] = 4$

Strong Data Structure

$A[0] = 6$

DS = Array int

$A[L \dots U]$

Lower

Upper

(maximum index)

① **Traversing:**

Print or reading etc.....

→ Input

Array A with n elements

→ Output

Depend on Procedure Traversal

→ Data structure

Int array with n elements

② while ($i \leq U$)

{ process($A[i]$);

$i++$;

End

: Big $O(n)$

◎ Sorting:

Input : An Array

Output : Sorted Array

D.S = Int Array $A[L \dots U]$

→ Bubble sort, Linear sort,
Insertion sort.

→ Bubble Sort:

L	0	1	2		U		
	6	5	4	3	2	1	

⇒ $i = U$

while ($i \geq L$)

{ $j = L$

while ($j < i$)

∴ swap

a b T

{ if ($a[j] > a[j+1]$)

{ swap($a[j], a[j+1]$);

$j++$;

$i = i - 1$;

∴ Big $O(n^2)$

→

	L	0	1	2		U
	6	5	4	3	2	1

j ↑

	5	6	4	3	2	1

5 4 6 3 2 1

5 4 3 6 2 1

5 4 3 2 6 1

first loop completed

5 4 3 2 1 6

→ 4 5 3 2 1 6

4 3 5 2 1 6

4 3 2 5 1 6

4 3 2 1 5 6

4 3 2 1 5 6

: Loop completed
then
subtract
upper limit
and start
from
lower limit.

→ 4 3 2 1 5 6

3 4 2 1 5 6

3 2 4 1 5 6

3 2 1 4 5 6

→ 3 2 1 4 5 6

2 3 1 4 5 6

2 1 3 4 5 6

→ 2 1 3 4 5 6

1 2 3 4 5 6

Sorted

→ Search Array (Key):

Input: Key passes in Array to be searched

Output: Index of Key

D.S : Array int $A[L \dots U]$

→ $i = 1$, found = 0, index = -1;

while ($i \leq U$)

{ if ($a[i] == \text{Key}$)

{ found = 1;

index = i;

}

i++;

return index;

→ Insert element:

6	5	3	2	1	
---	---	---	---	---	--

↓

→ If ($U == \text{capacity}$)

{ return;

}

$i = U$

while ($i \geq \text{location}$)

{ $A[i+1] = A[i]$;

$i--$;

$A[\text{index}] = E;$

$U = U + 1;$

→ Remove element:

6	4	3	2	4
---	---	---	---	---

$\text{found} = 0$
 $\text{for } (\text{int } i = 0; i < \text{size}; i++)$ $\text{ss found} = 0$ $\uparrow$ index Key

$\{$
 $\text{if } (a[i] == \text{Key})$
 $\text{while } (j < \text{size})$
 $\{ a[i] = a[i+1]$
 $j++;$
 $\}$
 $\}$

→ $i = U;$
 $\text{while } (i < U)$

$\{ a[i] = a[i+1];$
 $i--;$
 $\}$

⊛ Merge Arrays (A_1, A_2)

$A_1 = L_1 \text{ --- } U_1$

$A_2 = L_2 \text{ --- } U_2$

A_1

0	1	2	3	4
---	---	---	---	---

A_2

0	1	2	3
---	---	---	---

Soln:- $\text{Array } a = \text{new Array}[U_1 + U_2]$

$\text{for } (\text{int } i = 0; i < U_1; i++)$

$\{ a[i] = A_1[i];$

$\}$
 $\text{for } (\text{int } j = 0; j < U_2; j++)$

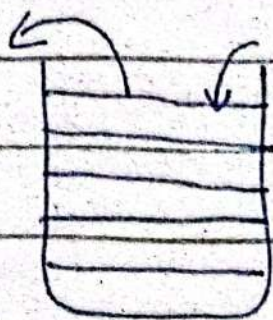
$\{ a[i] = A_2[j];$

() Array sorted by putting higher element of array in starting index.

◎ STACK:

Last In First out

(LIFO)



: push (insert data in stack)

: pop (remove data from stack)

→ Stack is managed through array but any central data can't be taken in.

→ stack pointer (points the top of stack).



Push A

Push B

Push C

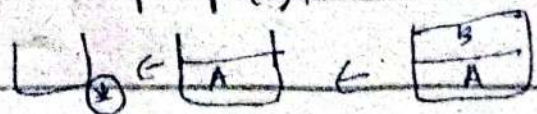
Push D

pop();

pop();

pop();

pop();



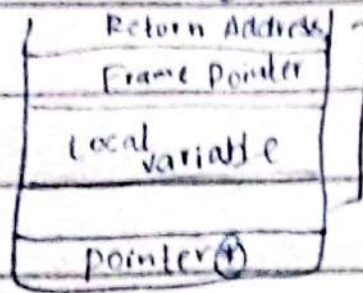
() All operations are constant time operations in stack ADT

: Big $O(1)$.

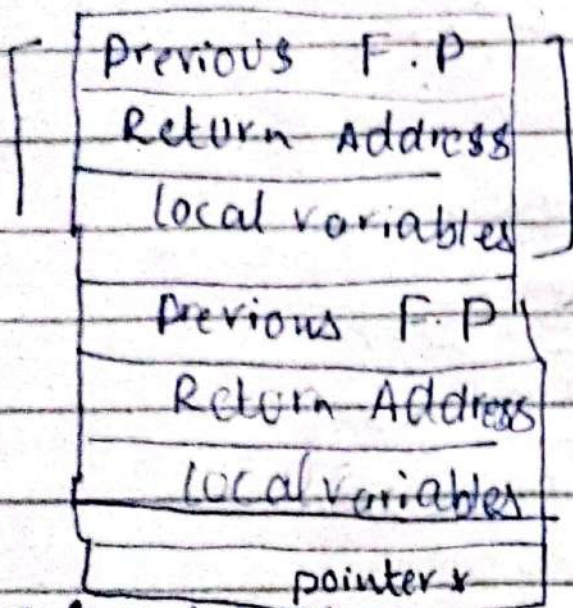
: ctrl Z eg

(Undo)
Plates

* System Stack:



- Recursion calls
- Frame Pointer increment
- After call
- Frame Pointer decrement



→ Activation frame stack frame

27/2/24

◎ STACK:

PUSH(A);

POP();

Recursion (calling of function).

* Function recursion:

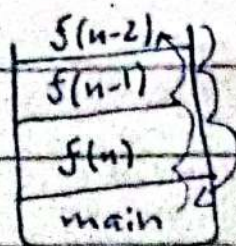
① - Itself

② - changed/different parameter

(calls function by changing condition). (calls itself by changing parameter).

$$2^3 = 2^2 \cdot 2^1$$

*



(When base calls then it will return to that function called).

* Base case are important for writing

in recursive function.

⑧ Stack

```
{ int *stack
  int capacity;
  int top;
public:
  Stack();
  is Empty();
  is Full();
  int Top();
  int Pop();
  void Push(int s);
}
```

: top pointer
acts as
function
pointer

: made for
template
class
(char stack
etc...)

(top most
element
shows)
(return top most
and also
remove it)

→ Stack(int capacity)

```
{ if (capacity < 0)
```

```
{ return 0;
```

```
  printOR capacity > 0;
```

```
  Stack = new int [capacity];
```

```
  top = -1;
```

is Empty

Top == -1

is Full

(Top != -1)

→ bool is Empty()

```
{ return top == -1;
}
```

→ bool is Full()

```
{ return top == capacity;
}
```

→ int Top()

```
{ if (is Empty)
  { return -1;
  }
```

```
  return stack[Top];
}
```

: ADT
set
top by
itself

: push &
pop
only
handle
top.

→ int Pop()

```
{ if (is Empty)
```

```
  { return -1;
  }
```

else

```
{ int item = stack[Top];
```

```
  Top = Top - 1;
```

```
  return item;
}
```

: insert:
top + 1
then
add.

AA : Dry Run + Conceptual
★ Paper

: all operations
are
constant
time
operations.

```
→ void Push(int e)
    {
        if (is Full)
        {
            Print "Stack is Full";
        }
        else
        {
            top = top + 1;
            stack[top] = e;
        }
    }
}
```

⊛ Dry - Run:

capacity = 4

top = ~~-1~~ ~~2~~ ~~1~~ ~~0~~ ~~1~~ ~~2~~ ~~3~~

top = -1

→ Push(int) → top = 0

Push(int) → top = 1

Push(int) → top = 2

pop() → top = 1

pop() → top = 0

Push() → top = 1

Push() → top = 2

Push() → top = 3

pop() → top = 2

pop() → top = 1

pop() → top = 0

pop() → top = -1

pop() → top = -1

return -1
as stack
is Empty.

→ Application:

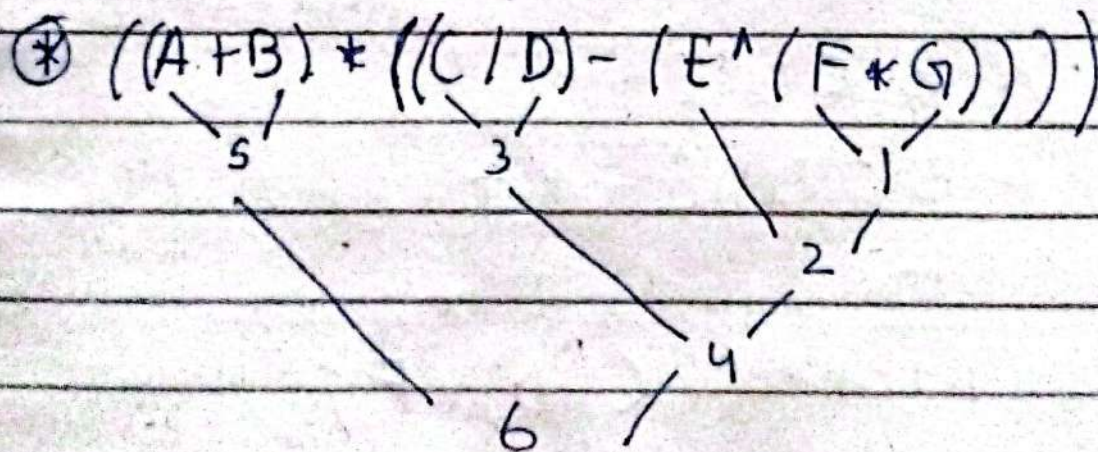
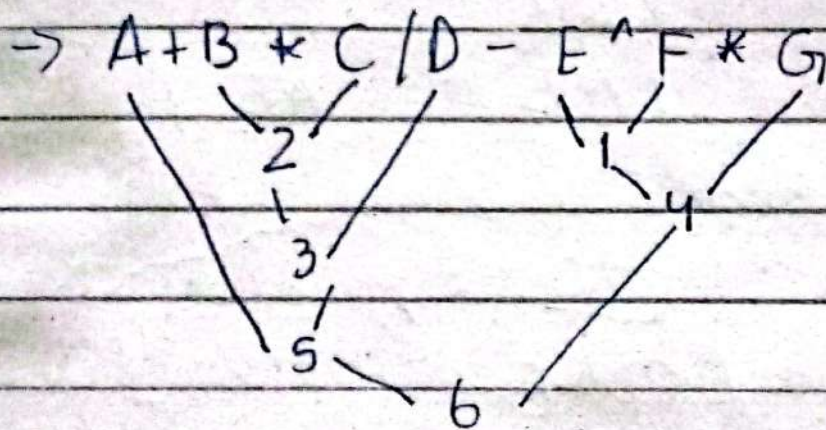
⊛ Evaluating arithmetic expression

⊛ Recursive / Recursion

⊛ Arithmetic (Mathematical expression)
(BODMAS Rule).

→ $A + D - C \times D$, $A * B - C / D$

→ Precedence (Exponent, Divide etc...)



⊛ Lab Stack (General Application)

E.g: Palindrome etc. — Any

(101), (DAD)
(MOM)

5/3/24

(*) Sparse Matrix multiplication.
(ADT).

(*) Precedence (left to right)
($\wedge$, $*$, $/$, $+$, $-$).

(*) Infix	Prefix	Postfix
$A - B$	$-AB$	$AB -$
$A + B$	$+AB$	$AB +$
$A * B$	$*AB$	$AB *$
A / B	$/AB$	$AB /$

(*) Postfix	Prefix
$A + B * (C^{\wedge} D - E)$	$A + B * (^{\wedge} C D - E)$
$A + B * (C D^{\wedge} E -)$	$A + B * (-^{\wedge} C D E)$
$A + B C D^{\wedge} E - *$	$+ A * B - ^{\wedge} C D E$
$\rightarrow A B C D^{\wedge} E - * +$	

: operator
stack
: value
stack.

(*) $((A + B) * ((C^{\wedge} D) - F))$

Postfix

$(AB + * (CD^{\wedge} - F))$

$(AB + * CD^{\wedge} F - *)$

$(AB + CD^{\wedge} F - *)$

$$(*) (+AB * (^{CD} - E))$$

$$(+AB * ^{CD} - E)$$

Prefix

$$+AB - ^{CDE}$$

$$(*) A/B - C + D * F - A * C$$

Postfix

$$A/B C - + D F * - A C *$$

$$(*) \underline{A B C - / D F * + A C * -}$$

$$✓ (*) - + - / A B C * D F * A C$$

Prefix

$$(*) A B - C / + D F * - A C *$$

$$(*) \underline{A B C / - D F * + A C * -}$$

Postfix

$$✓ \underline{A B / C - D F * + A C * -}$$

→ Parenthesized :

$$(((A/B) - C) + (D * F)) - (A * C)$$

(*) Sparse Matrix Multiplication

Assignment (Transpose, Multiplication, Addition)

(All functions important for lab).

7/3/24

④ Infix Operations:

- (1) Input arithmetic operations.
- (2) Read char from left to right.
(If oper ~~is~~ then display it).

→ If operator precedence is less than stack operator precedence then display.

→ Steps:

- ① → Scan expression from left to right.
- ② → If scanned character is an operand, then output else if scanned character is operator and if the precedence of scanned character is greater than the operator on the stack or the stack is empty or there is an opening parentheses on top of stack, then push

that character on the stack.

(3) → Pop all operators from the stack having precedence greater than equals to the precedence of scanned character.

(4) → If the scanned character is opening parentheses, then push it on stack and if the scanned character is closing, pop the character from the stack until opening parentheses present and then discard it.

(5) Output the remaining operators from the stacks if any.

① Example:

$a * (b + c + d)$

$a * (bc + d)$

$a * bc + d +$

$abc + d + *$

(Postfix)

	Stack	O/p
a	#	a
*	# *	a
(	# * (	a
b	# * (	ab
+	# * (+	ab
c	# * (+	abc +
+	# * (+	abc +
d	# * (+	abc + d +
)	# *	abc + d +
		abc + d + k

$$\rightarrow A + B * C / D - F + A \wedge E$$

Sol:-

$$A + B * C / D - F + A E^{\wedge}$$

$$\times \left[\begin{array}{l} A + B * C / D - F A E^{\wedge} + \\ A + B * C / D F A E^{\wedge} + \end{array} \right.$$

$$A + B * C D / - F + A E^{\wedge}$$

$$: ABC * D / +$$

$$F - A E^{\wedge}$$

$\rightarrow$

	Stack	O/p
A	#	A
+	# +	A
B	# +	AB
*	# + *	AB
C	# + *	ABC
/	# + /	ABC *
D	# + /	ABC * D / +
-	# + / -	ABC *
F	# + / -	
+	# +	
A	# +	
^	# + ^	
E	# + ^	

$$\rightarrow (A+B) * D + E / (F + A * D) + C$$

Sol:-

$$(AB+) * D + E / (F + AD*) + C$$

$$(AB+) * D + E / (FAD*+) + C$$

$$AB+ * D + E / FAD*+ + C$$

$$AB+ * D + EFAD*+ / + C$$

$$AB + D * + EFAD * + / C +$$

$$ABD * + EFAD * + / C ++$$

	Stack	O/P
(	# (	
A	# (	A
+	# (+	A
B	# (+	AB
)	# (	AB+
*	# *	AB+
D	# *	AB+D
+		
E		
/		
(		
F		
+		
A		
*		
D		
+		
C		

$$1 + 2 + 3 \Rightarrow 123++$$

$$12 + 3 \Rightarrow \{ 123+$$

$$1 + 23 \Rightarrow \{ 123+$$

$$1 - 2 - 3 - ++$$

$$\rightarrow \rightarrow \rightarrow$$

$$12 - 3 - +$$

$$1 - 23 +$$

* Infix $\rightarrow$ Prefix:

$\rightarrow$ Not equal precedence
(greater precedence)

① $\rightarrow$ Reverse expression

② $\rightarrow$ Nearly postfix.

③ $\rightarrow$ Reverse it, then prefix.

$$\rightarrow (A + B^{\wedge} C) * D + E^{\wedge} 5$$

SOL:-

$$\leftarrow 5^{\wedge} E + D * (C^{\wedge} B + A)$$

: Reverse

Postfix
(By token).

$$5E^{\wedge} + D * (CB^{\wedge} + A)$$

: converting
postfix

$$5E^{\wedge} + D * (CB^{\wedge} A +)$$

$$5E^{\wedge} + D CB^{\wedge} A + *$$

$$5E^{\wedge} D CB^{\wedge} A + * +$$

: Reversing

$$+ * + A^{\wedge} B C D^{\wedge} E 5$$

* Prefix and Postfix Evaluate
(one minor change)

① If postfix (left $\rightarrow$ right)
prefix (right $\rightarrow$ left)

② Operand then push to stack.

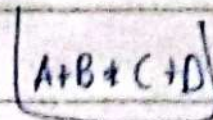
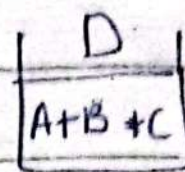
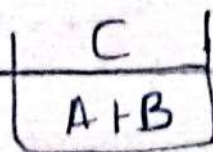
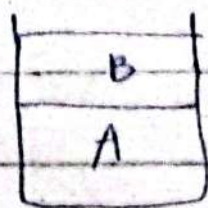
If operator then apply
operator on top most operands
and return result at stack.

→ (A+B) * (C+D) ^{prefix} → + * + A B C D

AB + * (C+D)

AB + C * + D

AB + C + D +



12 | 3 | 24

* Evaluation:

→ Postfix (one operand → second operand).

A B C D E

*: Practice

* Recursion:

The ability of the function to call itself.

→ Factorial:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

$$n! = n(n-1)(n-2)(n-3) \dots$$

$$n! = n(n-1)$$

$$\hookrightarrow n(n-1)(n-2)$$

$$\hookrightarrow n(n-1)(n-2)(n-3)$$

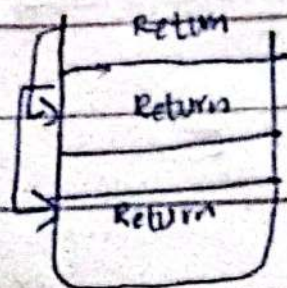
⊛ Base Case (Anchor / Ground case):

⊛ Inductive step

⊛ Factorial Iterating:

```
Fact = 1;  
for (int i = 0; i < N; i++)  
{ fact(N(i-1));  
}
```

⊛ Base Case is used as terminating point.



Backtracking

factorial

→ if $(n == 0 \mid n == 1)$

{ return 1 }

Base Case

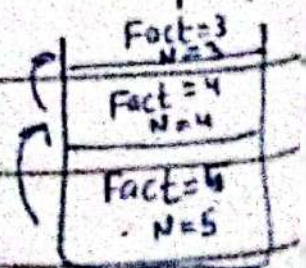
return fact(n-1); } → Inductive Step

⊛ Fact(5)

↳ Fact(5(5-1))

↳ Fact(4(4-1))

↳ Fact(3(3-1))



(*) Power function:

```
⊙  $x^N$   
result = 1;  
for (int i=0; i < N; i++)  
{ result = result * x;  
}
```

⊙ $x^4 \rightarrow x(x^3)$
 $\hookrightarrow x(x^2)$
 $\hookrightarrow x(x)$
 $\hookrightarrow x(x^0)$

calculate Pow (int N, int pow)

```
{ if (pow == 1)  
  { return N;  
  }  
  else if (pow == 0)  
  { return 1;  
  }
```

- If one function calls other (Direct Recursion)
- If a function calls itself without intervention of some other

function is called Direct recursion.

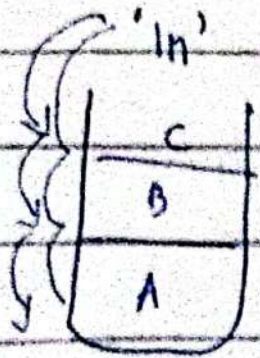
→ If a function itself with intervention of other function is called ^{indirect recursion} ~~can be converted into iterated solution~~.

→ Tail Recursion (Recursive function is the last call) (Recursive call).

→ Non-Tail Recursion (Recursive call is not last call). (Not be converted easily into Iterative function) (without stack).

E.g. void display()^{reverse}

(Non-Tail Recursion)



```
cin >> ch;
```

```
if (ch != '\n')
```

```
{ display reverse();
```

```
}
```

```
{ cout << ch;
```

```
}
```

```
}
```

Iterative

→ Stack <char> Reverse ()

```
{ stack <char> S;
```

```
char ch;
```

```
cin >> ch
```

```
while ( cin.get() != '\n' )
```

```
{ s.push(ch);
```

```
}
```

```
while ( !s.isEmpty() )
```

```
{
```

```
cout << s.pop();
```

```
}
```

```
}
```

: without
stack
Iterative
process
is not
possible

in Non-tail
Recursion.

⊕ Next class Quiz (Recursion)

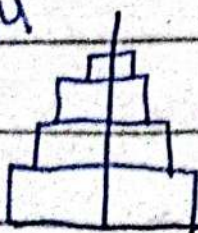
14/3/23

⊕ Assignment (Base Case + Inductive step)

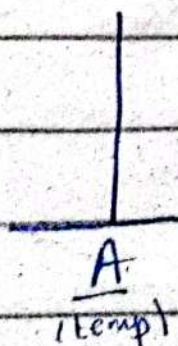
⊕ Space Complexity Increases of
Recursive Function but some problems
can't be solved by iteration.

→ Towers of Hanoi:

N=64



S



A
(temp)

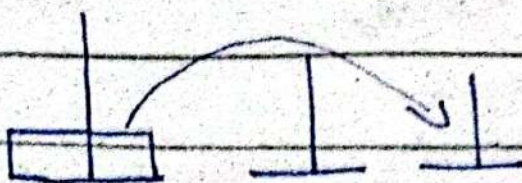


D

① Only top most
disk can be
moved

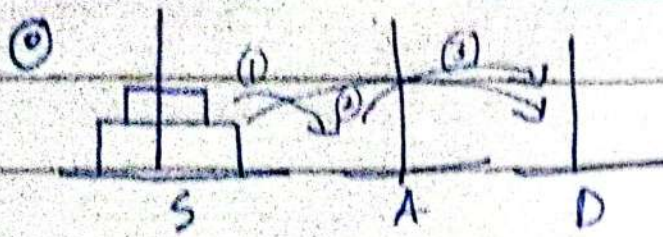
② Only smaller
disk

③ Only one
disk can be
moved at a
time



MOV S → D

Two disks



$N \rightarrow \text{Disk}$

$S \rightarrow A$ } $N-1$, Source To Aux (using Dest)
 $S \rightarrow D$ } 1, source to Dest
 $A \rightarrow D$ } $N-1$, Aux to Destination (using Source)

* $\odot$ $\text{HANOI}(N, S, A, D)$

{ if ($N == 1$) $\rightarrow$ Base case

{ move $S \rightarrow D$;

}

else $\rightarrow$ Inductive step

{ $\text{HANOI}(N-1, S, D, A)$;

Move $S \rightarrow D$

$\text{HANOI}(N-1, A, S, D)$;

}

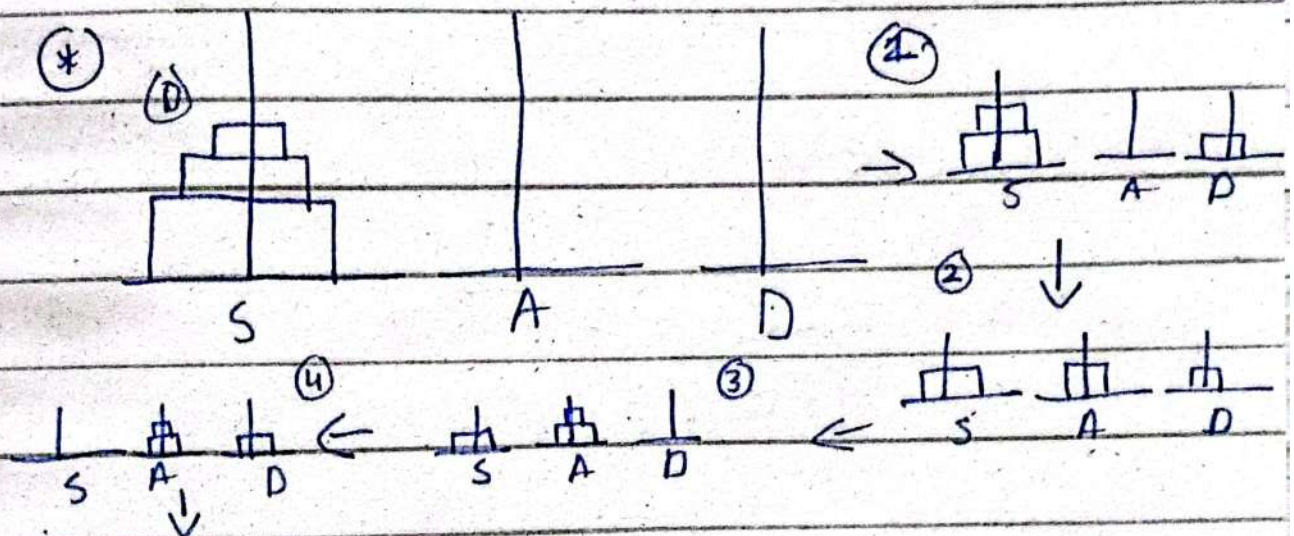
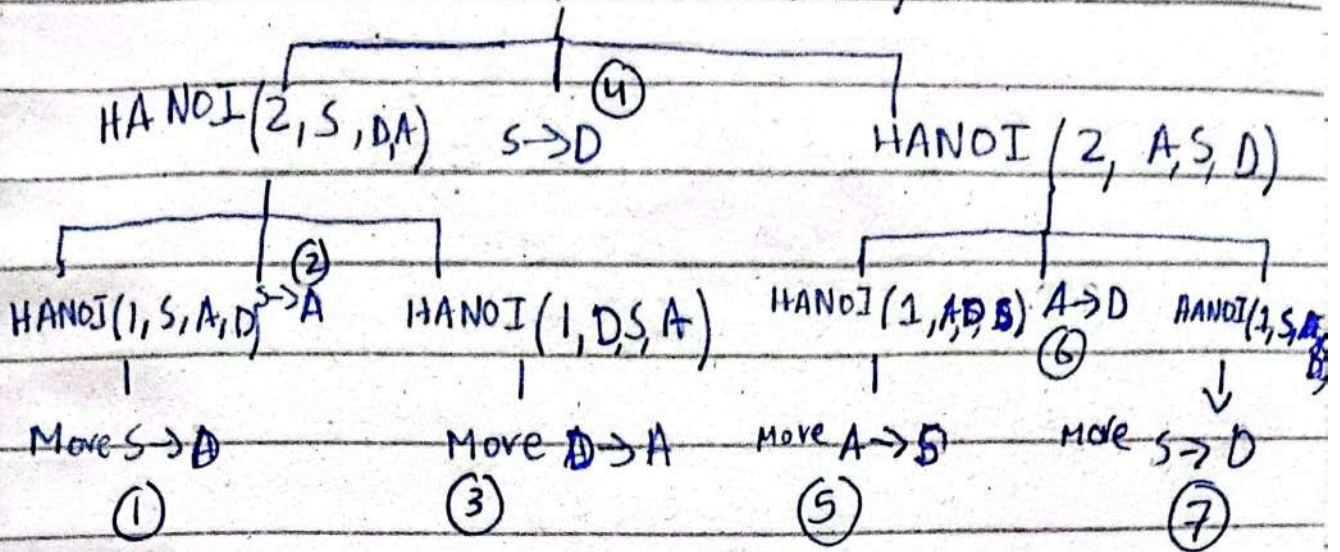
}

$\odot$ $\text{HANOI}(3, S, A, D)$

Assignment
: N=5 (HANOI)
(Dry Run and Tree)

Iterative
Solution

→ HANOI (3, S, A, D)

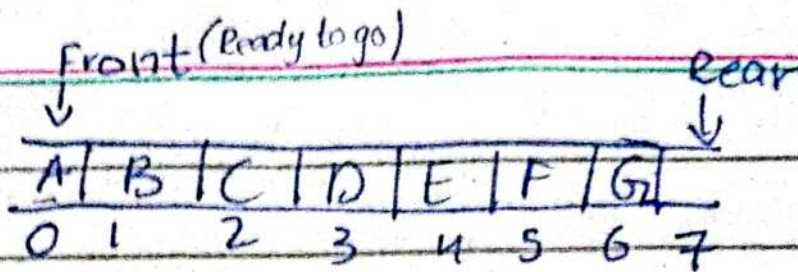


21/3/24

Linear Data Structure :

- ↳ Array
- ↳ Stack → LIFO
- ↳ Queue → ordered & Linear (FIFO).

① Front (1st element), Rear (where element to be added)



(*) Two pointers: front and rear are used in queue.

E.g.: (Print, Car Parking).

(*) ADT

```
{ Enqueue(e);
  Dequeue();
}
```

(Insert)

(Removes front)

∴ (Front and rear operations are possible not from center or any other)

(*) Queue

```
{ Queue[size];
  Rear = -1;
  Front = -1;
```

bool is Full()

```
{ if (Rear == size-1)
  { return true;
  }
  else
    return false;
}
```

bool is Empty()

```
{ if (Front == Rear == -1)
  { return true;
  }
  else
    return false;
}
```

```
void Enqueue (int val)
```

```
{  
    if (isFull())  
    {  
        Print "Queue is full";  
        return;  
    }
```

```
    else
```

```
    {  
        Rear = Rear + 1;  
        Queue[Rear] = val;
```

: Insertion
Rear
: Removal
Front

```
        if (Front == -1)
```

: Starting
condition.

```
        {  
            Front = 0;  
        }
```

```
    }
```

```
int Dequeue ()
```

```
{  
    if (isEmpty())
```

```
    {  
        Print "Queue is Empty";  
        Return;  
    }
```

```
    else
```

```
    {  
        element = Queue[Front];
```

```
        (Front = Front + 1);
```

```
        if (Front == Rear)
```

```
        {  
            Front = -1;  
            Rear = -1;
```

```
        }  
        else
```

```
        {  
            Front = Front + 1;
```

```
        }  
        return element;
```

F R
↓
A

* Enqueue(A)
 Enqueue(B)
 " (C)
 " (D)
 " (E)

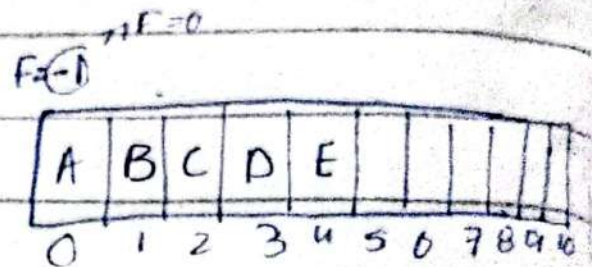
Dequeue()

Dequeue()

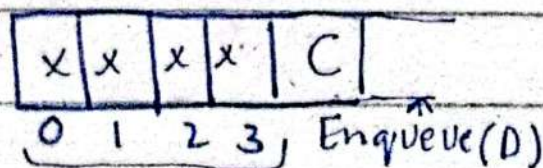
Dequeue()

Dequeue()

Dequeue()



* Issue:



Error: Full

* int shifted_dequeue()

{

* else

{ Front = Front + 1; }) *

else

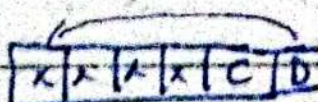
{ for (i=0; i <= Rear; i++)

{ Queue[i] = Queue[i+1]; }

Rear = Rear - 1;

}

: Next class
Circular Enqueue



: Front will never go next to rear.

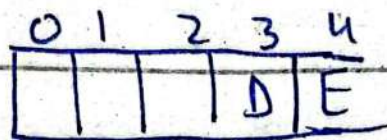
26/3/24

(*) Paper Mid:

- ADT's (Array, Stack, Queue)
- 10-15% code + dry Run/Structure
- Recursion (Tower of Hanoi)
- Syllabus (till Queue).

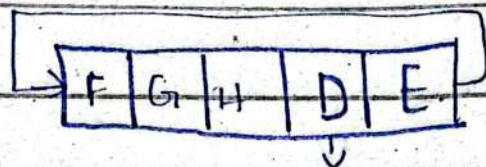
(*) Static DS (Array), Dynamic (Linked list)

(*) **Circular Queue:**



Enqueue(F) x
(error) (Full)

So we take circular queue
to solve issue.



Enqueue(F), (G), (H)

→ By taking mod we can solve it:

$$0 \% 5 = 0, \quad 1 \% 5 = 1, \quad 2 \% 5 = 2$$

$$3 \% 5 = 3, \quad 4 \% 5 = 4, \quad 5 \% 5 = 0$$

$$\boxed{\text{node} = i = (i+1) \% 5}$$

$$i=0 \rightarrow 1, \quad i=1 \rightarrow 2, \quad i=2 \rightarrow 3 \dots$$

$$i=5 \rightarrow 0 \dots$$

→ bool is Empty()

```
{  
  if (Front == Rear == -1)  
  {  
    Return true;  
  }  
  Return false;  
}
```

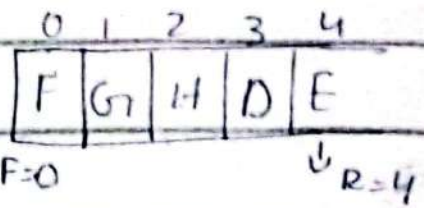
→ is Full { Front == 0 & Rear == size-1
Rear < Front }

bool isFull()
(F == 0 & R == size-1)

{ if (Front == (Rear+1) % size) F=0

{ Print "Full";

}



→ void Enqueue(e)

{ if (isFull())

{ Print "Queue is Full";

else

{ if (F == -1)

{ F = 0; }

Rear = (Rear+1) % size; change

Queue[Rear] = e;

}

→ int Dequeue()

{ if (is Empty)

{ return; (Print)

else

e = Queue[Front]

if (Front == Rear)

{ F == R == -1

}

else

Front = (Front + 1) % size; // change

}

}

→ **Priority Queue** (Printing Higher Authority first and then students).

elements

priorities

A

B

P₁

P₂

P₃

* : Included

in Exam

* : Kindly Practice

* : Lab included